كلية الهندسة بشبرا

FACULTY OF ENGINEERING AT SHOUBRA

Computer Organization

Snake Game Project

Group : 6

Islam Ahmed

Gehad Amr

Nada Ashraf

Nesreen Ahmed

Nour Abdel Sater

Dr. LAMIA.ALREFAAI

2025

Table of Contents

Abstract	**1**
Introduction	**1**
System Design Overview	**1**
Code Structure and Functions	**2**
Code Explanation	**4**
Results	**27**
Conclusion	**28**

Abstract

This report presents the design and implementation of the classic Snake game using MIPS Assembly language. The project demonstrates key computer architecture concepts, such as direct memory access, frame buffer manipulation, and low-level control flow. It integrates core features of the Snake game, including movement, growth, collision detection, and apple generation using system calls.

Introduction

The Snake game is a well-known and widely implemented arcade-style game, often used as a programming exercise to demonstrate logic implementation, state management, and graphics rendering. In this project, the game was developed entirely using **MIPS Assembly language**, offering a low-level, hardware-near programming experience that emphasizes how computation and memory operations are handled at the architectural level.

This project was conducted as part of the **Computer Organization** course at **Shoubra Faculty of Engineering**, with the primary objective of reinforcing fundamental concepts such as register usage, stack management, memory addressing, and low-level control flow. By implementing Snake in MIPS Assembly, this project provides hands-on experience with system-level programming while also improving understanding of how graphical output can be managed directly through memory (i.e., using a frame buffer).

Key aspects of the project include:

- Manual manipulation of memory locations to render pixels on a bitmap display.
- Use of **syscall services**, particularly for generating random numbers and managing the program's control flow.
- Careful use of registers and the **stack** to implement subroutine calls and maintain program stability.
- Logical conversion from 2D game coordinates to memory offsets for rendering and collision detection.

The project was developed using the **MARS (MIPS Assembler and Runtime Simulator)** tool, which provides support for a simulated bitmap display, facilitating visual feedback for the game's graphics.

In summary, this project not only delivers a working version of the Snake game but also serves as a proof of concept for how even high-level behaviors like real-time movement and randomized apple placement can be implemented using the limited and explicit operations available in assembly language.

System Design Overview

Describe the general architecture of the game.

1 Game Components :

- Snake Head and Body
- Apple
- Frame Buffer (Bitmap display)
- Input Handling (if included)

2 Memory Mapping :

- Where is the snake/apple stored?
- How is the frame buffer used?
- Key memory-mapped variables (e.g., xPos, yPos, appleX, appleY)

Code Structure and Functions

Figure 1

Main Section Flowchart

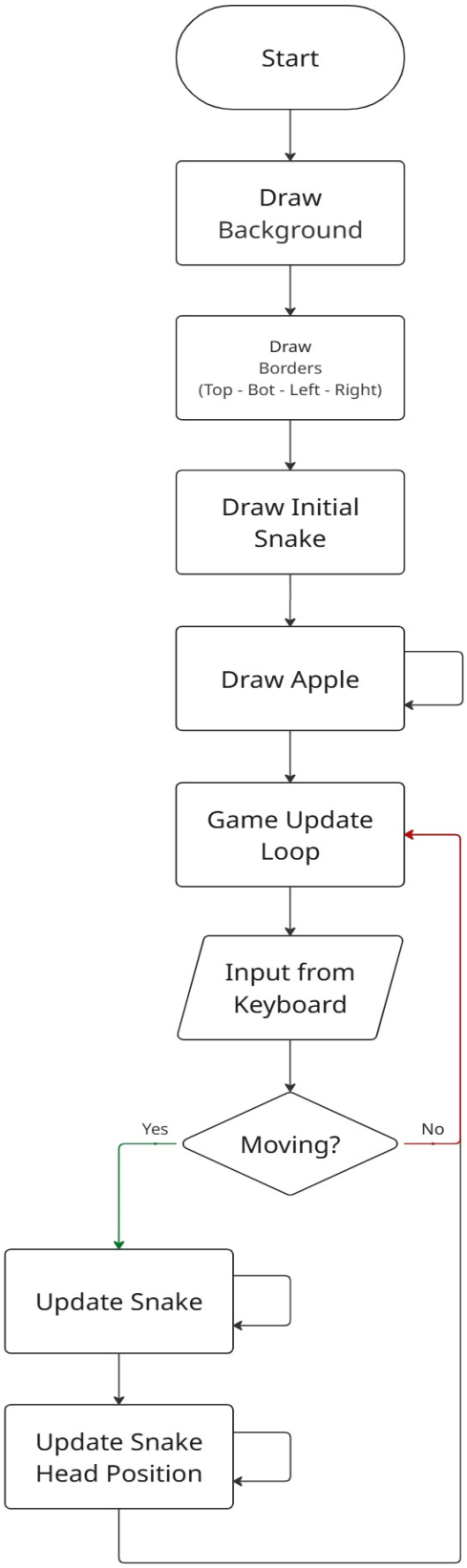


Figure 2

Update Snake Function Flowchart

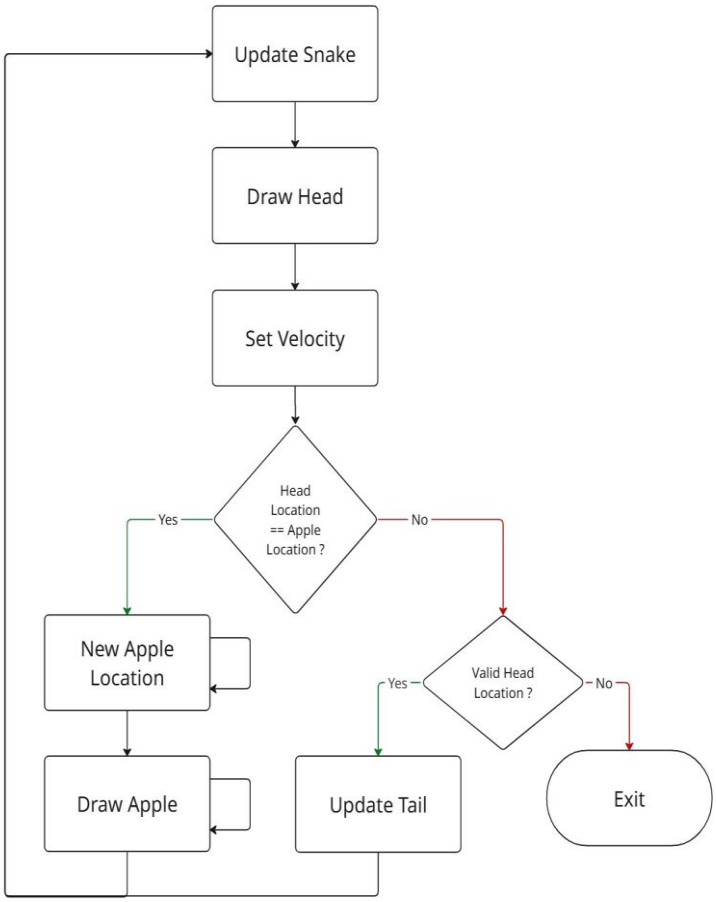
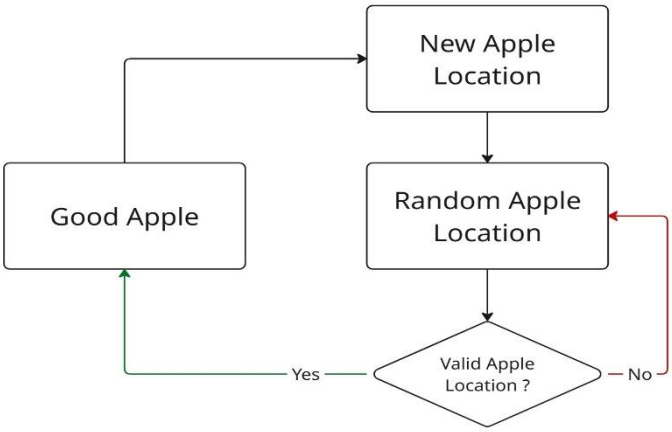


Figure 3

New Apple Location Function Flowchart



Code Explanation

I. .data section :

◆ framebuffer : .space 0x80000

- **Purpose**: Reserves memory for the game's **bitmap display**.
- **Size**: 0x80000 bytes = 524,288 bytes = enough to store **512 × 256 pixels**, where each pixel is 4 bytes (32-bit color).

◆ xVel and yVel: .word 0

- **Purpose**: Store the **current velocity** of the snake in the X and Y directions.
- **Initial Value**: 0 (snake starts stationary).

◆ xPos: .word 50, yPos: .word 27

- **Purpose**: Represent the **current position of the snake's head** on the grid.
- **Values**: (50, 27) — this is in grid units (not pixels), where the total grid size is 64 × 32.

◆ tail: .word 7624

- **Purpose**: Index in the bitmap memory where the **tail** of the snake currently is.
- Used to **clear** the tail's previous position as the snake moves.

◆ appleX: .word 32, appleY: .word 16

- **Purpose**: Represent the **apple's current location** in grid coordinates.
- Initial value places it roughly in the center of the screen.

◆ snakeUp / snakeDown / snakeLeft / snakeRight

- **Purpose**: These are **unique 32-bit colors** used to:
  - **Display** the snake in different directions.
  - **Encode direction** as part of the color value so that game logic can **decode** the direction from the bitmap directly.

For example, 0x0000ff00 is green for **up**, and the first byte is used to **identify direction**.

◆ xConversion: .word 64

- **Purpose**: Constant used to convert the y coordinate to the row offset (because each row has 64 cells, i.e., 512 pixels / 8 pixels per cell).

◆ yConversion: .word 4

- **Purpose**: Constant used to convert the (x, y) cell index to the actual memory offset (because each cell = 4 bytes).

II. .text section :

1. ### DRAW BACKGROUND SECTION

```
### DRAW BACKGROUND SECTION

        la      $t0, frameBuffer        # load frame buffer addres
        li      $t1, 8192               # save 512*256 pixels  (4*64*32)  ((512*256 /(8*8) )*4))


        li      $t2, 0x00d3d3d3         # load light gray color
l1:
        sw      $t2, 0($t0)
        addi    $t0, $t0, 4     # advance to next pixel position in display
        addi    $t1, $t1, -1    # decrement number of pixels
        bnez    $t1, l1         # repeat while number of pixels is not zero
```

Explanation

1. **Load Frame Buffer Address**

   `la $t0, frameBuffer`

   Load the starting memory address of the frame buffer into register `$t0`. This is where the pixel data for the screen begins.

2. **Set Loop Counter**

   `li $t1, 8192`

   Initialize the counter `$t1` to 8192. This represents the total number of pixels that need to be drawn (calculated from screen dimensions and bytes per pixel).

3. **Load Background Color**

   `li $t2, 0x00d3d3d3`

   Load the color value for the background (light gray) into register `$t2`.

4. **Start Loop (Label: l1)**

   `l1:` marks the beginning of the loop that fills the frame buffer.

5. **Store Background Color in Current Pixel**

   `sw $t2, 0($t0)`

   Store the background color value at the memory address pointed to by `$t0` (current pixel location).

6. **Move to Next Pixel**

   `addi $t0, $t0, 4`

   Increment the pixel pointer `$t0` by 4 bytes to move to the next pixel in the frame buffer.

7. **Decrement Loop Counter**

   `addi $t1, $t1, -1`

   Decrease the pixel count `$t1` by 1.

8. **Repeat Loop if Pixels Remain**

   `bnez $t1, l1`

   Branch back to `l1` if the pixel counter `$t1` is not zero, continuing the loop to fill all pixels.

2. ### DRAW BORDER SECTION

```
        # top wall section
        la      $t0, frameBuffer        # load frame buffer addres
        addi    $t1, $zero, 64          # t1 = 64 length of row  (512 'width of all row' / 8 'width of row' )
        li      $t2, 0x00000000         # load black color
drawBorderTop:
        sw      $t2, 0($t0)             # color Pixel black
        addi    $t0, $t0, 4             # go to next pixel
        addi    $t1, $t1, -1            # decrease pixel count
        bnez    $t1, drawBorderTop      # repeat unitl pixel count == 0
```

Explanation

1. **Load Frame Buffer Address**

   `la $t0, frameBuffer`

   Load the starting address of the frame buffer into register `$t0`. This points to the beginning of the screen memory where pixels are stored.

2. **Set Pixel Count for One Row**

   `addi $t1, $zero, 64`

   Initialize register `$t1` to 64. This represents the number of pixels in the top row (calculated from screen width and pixel grouping).

3. **Load Wall Color (Black)**

   `li $t2, 0x00000000`

   Load the color black into register `$t2`, which will be used to draw the top wall.

4. **Start Drawing Loop (Label: drawBorderTop)**

   `drawBorderTop:` marks the beginning of the loop that paints the top row.

5. **Paint Current Pixel Black**

   `sw $t2, 0($t0)`

   Store the black color value at the current pixel memory location pointed to by `$t0`.

6. **Move to Next Pixel**

   `addi $t0, $t0, 4`

   Increment `$t0` by 4 bytes to point to the next pixel position.

7. **Decrement Pixel Counter**

   `addi $t1, $t1, -1`

   Decrease the pixel count `$t1` by 1.

8. **Repeat Loop Until Done**

   `bnez $t1, drawBorderTop`

   If `$t1` is not zero, branch back to `drawBorderTop` to continue drawing the row until all 64 pixels are colored black.

```
        # Bottom wall section
        la      $t0, frameBuffer        # load frame buffer addres
        addi    $t0, $t0, 7936          # set pixel to be near the bottom left  (4*64*31)
        addi    $t1, $zero, 64          # t1 = 512 length of row

drawBorderBot:
        sw      $t2, 0($t0)             # color Pixel black
        addi    $t0, $t0, 4             # go to next pixel
        addi    $t1, $t1, -1            # decrease pixel count
        bnez    $t1, drawBorderBot      # repeat unitl pixel count == 0

        # left wall section
        la      $t0, frameBuffer        # load frame buffer address
        addi    $t1, $zero, 256         # t1 = 512 length of col
```

Explanation

1. **Load Frame Buffer Address**

   `la $t0, frameBuffer`

   Load the starting address of the frame buffer into register `$t0`.

2. **Set Pointer to Bottom Row Start**

   `addi $t0, $t0, 7936`

   Move `$t0` forward by 7936 bytes to point near the start of the bottom row on the screen.

   - Calculation: 4 bytes per pixel × 64 pixels per row × 31 rows down = 7936 bytes

     This positions `$t0` to the first pixel of the 32nd row (rows indexed from 0).

3. **Set Pixel Count for One Row**

   `addi $t1, $zero, 64`

   Initialize `$t1` to 64, representing the number of pixels in the bottom row.

4. **Draw Loop Start (Label: drawBorderBot)**

   Marks the beginning of the loop that draws the bottom wall.

5. **Paint Current Pixel Black**

   `sw $t2, 0($t0)`

   Store the color black (expected preloaded in `$t2`) into the current pixel location pointed to by `$t0`.

6. **Advance to Next Pixel**

   `addi $t0, $t0, 4`

   Increment `$t0` by 4 bytes to move to the next pixel.

7. **Decrement Pixel Counter**

   `addi $t1, $t1, -1`

   Decrease the pixel count `$t1` by 1.

8. **Loop Until All Pixels Drawn**

   `bnez $t1, drawBorderBot`

   Branch back to `drawBorderBot` if `$t1` is not zero, repeating until the entire bottom row is painted black.

```
        # left wall section
        la      $t0, frameBuffer        # load frame buffer address
        addi    $t1, $zero, 256         # t1 = 512 length of col

drawBorderLeft:
        sw      $t2, 0($t0)             # color Pixel black
        addi    $t0, $t0, 256           # go to next pixel (4*64) to go to the next cell in colom
        addi    $t1, $t1, -1            # decrease pixel count
        bnez    $t1, drawBorderLeft     # repeat unitl pixel count == 0
```

Explanation

1. **Load Frame Buffer Address**

   `la $t0, frameBuffer`

   Load the base address of the frame buffer into register `$t0`.

2. **Set Pixel Count for Column**

   `addi $t1, $zero, 256`

   Set `$t1` to 256, which is the number of pixels in the left column to draw.

   (Explanation: The left wall spans 256 pixels vertically.)

3. **Draw Loop Start (Label: drawBorderLeft)**

   Beginning of the loop that paints the left wall.

4. **Paint Current Pixel Black**

   `sw $t2, 0($t0)`

   Store the black color (assumed already loaded into `$t2`) at the current pixel address `$t0`.

5. **Advance to Next Pixel Down**

   `addi $t0, $t0, 256`

   Increase `$t0` by 256 bytes to move down one pixel vertically in the frame buffer.

   (Explanation: 256 bytes = 4 bytes per pixel × 64 pixels per row — advancing one row down in the same column.)

6. **Decrement Pixel Counter**

   `addi $t1, $t1, -1`

   Decrement the counter `$t1` by 1.

7. **Loop Until All Pixels Drawn**

   `bnez $t1, drawBorderLeft`

   Branch back to `drawBorderLeft` if `$t1` is not zero, continuing until all 256 pixels in the column are painted.

```
        # Right wall section
        la      $t0, frameBuffer        # load frame buffer address
        addi    $t0, $t0, 508           # make starting pixel top right
        addi    $t1, $zero, 255         # t1 = 512 length of col

drawBorderRight:
        sw      $t2, 0($t0)             # color Pixel black
        addi    $t0, $t0, 256           # go to next pixel
        addi    $t1, $t1, -1            # decrease pixel count
        bnez    $t1, drawBorderRight    # repeat unitl pixel count == 0
```

Explanation

1. **Load Frame Buffer Address**

   `la $t0, frameBuffer`

   Load the base address of the frame buffer into `$t0`.

2. **Set Starting Pixel for Right Wall**

   `addi $t0, $t0, 508`

   Offset `$t0` by 508 bytes to point near the top-right pixel of the screen.

   (Explanation: 508 bytes = (4 bytes × 64 pixels × 2 rows) - 4 bytes to move left one pixel)

3. **Set Pixel Count for Column**

   `addi $t1, $zero, 255`

   Set `$t1` to 255, the number of pixels to draw for the right wall vertically.

4. **Draw Loop Start (Label: drawBorderRight)**

   Beginning of the loop to paint the right wall.

5. **Paint Current Pixel Black**

   `sw $t2, 0($t0)`

   Store the black color at the pixel address `$t0`.

6. **Advance to Next Pixel Down**

   `addi $t0, $t0, 256`

   Move `$t0` down by 256 bytes to the next pixel in the same column, one row below.

7. **Decrement Pixel Counter**

   `addi $t1, $t1, -1`

   Decrease the pixel counter `$t1` by 1.

8. **Loop Until All Pixels Drawn**

   `bnez $t1, drawBorderRight`

   Loop back if pixels remain to be drawn.

3- ### draw initial snake section

```
### draw initial snake section
la      $t0, frameBuffer        # load frame buffer address
lw      $s2, tail               # s2 = tail of snake
lw      $s3, snakeUp            # s3 = direction of snake

add     $t1, $s2, $t0           # t1 = tail start on bit map display
sw      $s3, 0($t1)             # draw pixel where snake is
addi    $t1, $t1, -256          # set t1 to pixel above
sw      $s3, 0($t1)             # draw pixel where snake currently is
#addi   $t1, $t1, -256          # set t1 to pixel above
#sw     $s3, 0($t1)             # draw pixel where snake currently is

### draw initial apple
jal     drawApple
```

Explanation

1. **Load Frame Buffer Address**

   `la $t0, frameBuffer`

   Load the base address of the frame buffer into register `$t0`.

2. **Load Snake Tail Position**

   `lw $s2, tail`

   Load the memory address or offset of the snake's tail into register `$s2`.

3. **Load Snake Direction Color**

   `lw $s3, snakeUp`

   Load the color or pattern representing the snake's upward direction into `$s3`.

4. **Calculate Tail Pixel Address**

   `add $t1, $s2, $t0`

   Add the tail offset to the frame buffer base address, resulting in the actual pixel address in `$t1`.

5. **Draw Tail Pixel**

   `sw $s3, 0($t1)`

   Store the snake's color or pattern at the tail pixel address to visually render the tail.

6. **Calculate Pixel Above Tail**

   `addi $t1, $t1, -256`

   Move the pixel address `$t1` one row up in the frame buffer by subtracting 256 bytes (4 bytes per pixel × 64 pixels per row).

7. **Draw Next Snake Pixel Above Tail**

   `sw $s3, 0($t1)`

   Draw the snake color at the pixel directly above the tail to start building the snake body.

8. **Optional Additional Pixel Above (Commented Out)**

   The code for drawing an extra pixel above the second one is commented out but could be used to extend the initial snake length.

9. **Draw Initial Apple**

   `jal drawApple`

   Jump and link to the `drawApple` subroutine to draw the apple on the game board.

4- # This is the update function for game

```
gameUpdateLoop:
        lw      $t3, 0xffff0004         # get keypress from keyboard input

        ### Sleep for 66 ms so frame rate is about 15
        addi    $v0, $zero, 32  # syscall sleep
        addi    $a0, $zero, 66  # 66 ms
        syscall

        beq     $t3, 100, moveRight     # if key press = 'd' branch to moveright
        beq     $t3, 97, moveLeft       # else if key press = 'a' branch to moveLeft
        beq     $t3, 119, moveUp        # if key press = 'w' branch to moveUp
        beq     $t3, 115, moveDown      # else if key press = 's' branch to moveDown
        #beq    $t3, 0, moveUp          # start game moving up
        j gameUpdateLoop
```

Explanation

1. **Read Keyboard Input**

   `lw $t3, 0xffff0004`

   Load the key press value from a specific memory-mapped keyboard input register into `$t3`.

2. **Pause to Control Frame Rate**

   To maintain a frame rate of roughly 15 frames per second, the program pauses for approximately 66 milliseconds:

   ```
   addi $v0, $zero, 32  # Prepare syscall for sleep
   addi $a0, $zero, 66  # Set sleep time to 66 milliseconds
   syscall              # Execute sleep syscall
   ```

3. **Check Key Press and Branch to Direction Handling**

   The loop compares the key pressed (`$t3`) to ASCII values for movement keys and jumps to the corresponding movement routine:

   - If key is `'d'` (ASCII 100), jump to `moveRight`
   - If key is `'a'` (ASCII 97), jump to `moveLeft`
   - If key is `'w'` (ASCII 119), jump to `moveUp`
   - If key is `'s'` (ASCII 115), jump to `moveDown`

4. **Repeat Loop if No Valid Key Press**

   If none of these keys are pressed, the program jumps back to `gameUpdateLoop` to wait for the next key press and continue the game.

5- Movement Handling

```
moveUp:
        lw      $s3, snakeUp     # s3 = direction of snake
        add     $a0, $s3, $zero # a0 = direction of snake
        jal     updateSnake

        # move the snake
        jal     updateSnakeHeadPosition

        j       exitMoving
```

```
exitMoving:
        j       gameUpdateLoop          # loop back to beginning
```

Common Steps in All Movement Labels:

Each direction (`moveUp` , `moveDown` , `moveLeft` , `moveRight`) follows the same structure with only the direction register (`snakeUp` , `snakeDown` , etc.) changing.

`moveUp` (similar for other directions):

1. **Load the Direction Value**

   `lw $s3, snakeUp`

   Load the pre-defined memory value that represents "up" movement into `$s3` .

2. **Pass Direction as Argument to updateSnake**

   `add $a0, $s3, $zero`

   Move the direction value into `$a0` , which is the first argument register in MIPS.

3. **Update Snake's Body**

   `jal updateSnake`

   Call the `updateSnake` function, which updates the position of the entire snake body based on the direction.

4. **Update Snake's Head**

   `jal updateSnakeHeadPosition`

   After the body is updated, the head is moved to the new location.

5. **Return to Main Loop**

   `j exitMoving`

   Jump to `exitMoving` which loops back to `gameUpdateLoop` .

`exitMoving`

This line redirects control back to the main game loop so the game continues running and responds to future input.

6- # updateSnake Function 'Draw Snake Head'

🔄 Step 1: Set Up Stack Frame

```
addiu $sp, $sp, -24
sw    $fp, 0($sp)
sw    $ra, 4($sp)
addiu $fp, $sp, 20
```

Purpose:

- Reserve space on the stack
- Save return address and frame pointer
- Prepare to safely return after the function ends

📍 Step 2: Load Current Snake Position

```
lw $t0, xPos    # Load x position into $t0
lw $t1, yPos    # Load y position into $t1
```

Purpose:

- Get the coordinates of the snake's head

➗ Step 3: Convert 2D Position to 1D Index

```
lw    $t2, xConversion  # xConversion = 64 (screen width)
mult  $t1, $t2          # y * 64 (row offset)
mflo  $t3               # $t3 = y * 64
add   $t3, $t3, $t0     # $t3 = y * 64 + x
```

Purpose:

- Convert (x, y) to a 1D index in a linear frame buffer

✖️ Step 4: Convert Index to Byte Offset

```
lw    $t2, yConversion  # yConversion = 4 (bytes per pixel)
mult  $t3, $t2          # index * 4
mflo  $t0               # $t0 = byte offset for pixel
```

Purpose:

- Convert pixel index to memory offset in bytes

Step 5: Calculate Final Pixel Address and Draw

```
la    $t1, frameBuffer  # Load base address of display
add   $t0, $t1, $t0     # Final pixel address = base + offset
lw    $t4, 0($t0)       # Backup old pixel value (optional)
sw    $a0, 0($t0)       # Draw new pixel (snake head) using $a0
```

Purpose:

- Compute where in memory to draw
- Draw the snake's head with its direction and color

7- # updateSnake Function 'Set Velocity'

```
### Set Velocity
lw      $t2, snakeUp                    # load word snake up = 0x0000ff00
beq     $a0, $t2, setVelocityUp         # if head direction and color == snake up branch to setVelocityUp

lw      $t2, snakeDown                  # load word snake up = 0x0100ff00
beq     $a0, $t2, setVelocityDown       # if head direction and color == snake down branch to setVelocityUp

lw      $t2, snakeLeft                  # load word snake up = 0x0200ff00
beq     $a0, $t2, setVelocityLeft       # if head direction and color == snake left branch to setVelocityUp

lw      $t2, snakeRight                 # load word snake up = 0x0300ff00
beq     $a0, $t2, setVelocityRight      # if head direction and color == snake right branch to setVelocityUp
```

Explanation

1. Load `snakeUp` value into `$t2`.
2. Compare `$a0` with `snakeUp` → if equal, jump to `setVelocityUp`.
3. Load `snakeDown` value into `$t2`.
4. Compare `$a0` with `snakeDown` → if equal, jump to `setVelocityDown`.
5. Load `snakeLeft` value into `$t2`.
6. Compare `$a0` with `snakeLeft` → if equal, jump to `setVelocityLeft`.
7. Load `snakeRight` value into `$t2`.
8. Compare `$a0` with `snakeRight` → if equal, jump to `setVelocityRight`.

```
setVelocityUp:
        addi    $t5, $zero, 0           # set x velocity to zero
        addi    $t6, $zero, -1          # set y velocity to -1
        sw      $t5, xVel               # update xVel in memory
        sw      $t6, yVel               # update yVel in memory
        j exitVelocitySet
```

▲ `setVelocityUp`

1. Set `$t5 = 0` → `xVel = 0` (no horizontal movement)
2. Set `$t6 = -1` → `yVel = -1` (move up)
3. Store `$t5` into `xVel`
4. Store `$t6` into `yVel`
5. Jump to `exitVelocitySet`

```
setVelocityDown:
        addi    $t5, $zero, 0           # set x velocity to zero
        addi    $t6, $zero, 1           # set y velocity to 1
        sw      $t5, xVel               # update xVel in memory
        sw      $t6, yVel               # update yVel in memory
        j exitVelocitySet
```

▼ `setVelocityDown`

1. Set `$t5 = 0` → `xVel = 0`
2. Set `$t6 = 1` → `yVel = 1` (move down)
3. Store `$t5` into `xVel`
4. Store `$t6` into `yVel`
5. Jump to `exitVelocitySet`

```
setVelocityLeft:
        addi    $t5, $zero, -1          # set x velocity to -1
        addi    $t6, $zero, 0           # set y velocity to zero
        sw      $t5, xVel               # update xVel in memory
        sw      $t6, yVel               # update yVel in memory
        j exitVelocitySet
```

◀ `setVelocityLeft`

1. Set `$t5 = -1` → `xVel = -1` (move left)
2. Set `$t6 = 0` → `yVel = 0`
3. Store `$t5` into `xVel`
4. Store `$t6` into `yVel`
5. Jump to `exitVelocitySet`

```
setVelocityRight:
        addi    $t5, $zero, 1           # set x velocity to 1
        addi    $t6, $zero, 0           # set y velocity to zero
        sw      $t5, xVel               # update xVel in memory
        sw      $t6, yVel               # update yVel in memory
        j exitVelocitySet
```

▶ `setVelocityRight`

1. Set `$t5 = 1` → `xVel = 1` (move right)
2. Set `$t6 = 0` → `yVel = 0`
3. Store `$t5` into `xVel`
4. Store `$t6` into `yVel`
5. Jump to `exitVelocitySet`

8- ### Head location checks

```
exitVelocitySet:

        ### Head location checks
        li      $t2, 0x00ff0000         # load red color
        bne     $t2, $t4, headNotApple  # if head location is not the apple branch away

        jal     newAppleLocation
        jal     drawApple
        j       exitUpdateSnake
```

Explanation

1. **Load red color value** into `$t2` (0x00ff0000). This color likely represents the apple.
2. **Compare** `$t2` (apple color) with `$t4` (original pixel color at the snake's head position).
3. If **colors do not match** (`bne`), it means the snake head is **not on the apple**, so branch to `headNotApple`.
4. If colors **match** (snake head is on the apple):
   - Call `newAppleLocation` to generate a new apple position.
   - Call `drawApple` to draw the apple at the new location.
   - Jump to `exitUpdateSnake` to end the update process.

```
headNotApple:
        li      $t2, 0x00d3d3d3                 # load light gray color
        beq     $t2, $t4, validHeadSquare       # if head location is background branch away

        addi    $v0, $zero, 10  # exit the program
        syscall
```

Explanation

1. **Load light gray color** (0x00d3d3d3) into `$t2`. This color likely represents the background or empty space.
2. **Compare** `$t2` (background color) with `$t4` (original pixel color at the snake's head position).
3. If the colors **match** (head location is background), branch to `validHeadSquare` to continue the game (snake moves into a valid empty square).
4. If the colors **do not match**, it means the snake has collided with something else (like the wall or itself), so:
   - Prepare to exit the program by loading 10 into `$v0` (exit syscall).
   - Make the syscall to terminate the program (game over).

9- ### Remove Tail && ### update new Tail

```
validHeadSquare:

        ### Remove Tail
        lw      $t0, tail               # t0 = tail
        la      $t1, frameBuffer        # load frame buffer address
        add     $t2, $t0, $t1           # t2 = tail location on the bitmap display
        li      $t3, 0x00d3d3d3         # load light gray color
        lw      $t4, 0($t2)             # t4 = tail direction and color
        sw      $t3, 0($t2)             # replace tail with background color
```

Explanation

1. **Load tail position** from memory into `$t0`.
   (This is the current tail coordinate in the snake's body.)
2. **Load frame buffer base address** into `$t1`.
   (Starting point of the bitmap display memory.)
3. **Calculate tail pixel address:**
   Add tail offset `$t0` to frame buffer base `$t1`, result in `$t2`.
   (`$t2` now points to the exact pixel location of the tail in the framebuffer.)
4. **Load light gray color** (background color) into `$t3` (0x00d3d3d3).
   (This color represents clearing the pixel.)
5. **Load current pixel data** at tail position into `$t4`.
   (Read what color/direction data was there before.)
6. **Clear the tail pixel** by storing light gray color at the tail position in the framebuffer.
   (`sw $t3, 0($t2)` effectively removes the tail from the display.)

```
### update new Tail
lw      $t5, snakeUp                    # load word snake up = 0x0000ff00
beq     $t5, $t4, setNextTailUp         # if tail direction and color == snake up branch to setNextTailUp

lw      $t5, snakeDown                  # load word snake up = 0x0100ff00
beq     $t5, $t4, setNextTailDown       # if tail direction and color == snake down branch to setNextTailDown

lw      $t5, snakeLeft                  # load word snake up = 0x0200ff00
beq     $t5, $t4, setNextTailLeft       # if tail direction and color == snake left branch to setNextTailLeft

lw      $t5, snakeRight                 # load word snake up = 0x0300ff00
beq     $t5, $t4, setNextTailRight      # if tail direction and color == snake right branch to setNextTailRight
```

Explanation

1. **Load the color/direction code for "snake up"** into `$t5`.
2. **Compare tail pixel's direction/color** `$t4` **with "snake up" code** `$t5`**:**
   - If equal, jump to `setNextTailUp` to update the tail position upwards.
3. **Load "snake down" color/direction** into `$t5`.
4. **Compare tail pixel's** `$t4` **with "snake down" code:**
   - If equal, jump to `setNextTailDown` to update tail position downwards.
5. **Load "snake left" color/direction** into `$t5`.
6. **Compare tail pixel's** `$t4` **with "snake left" code:**
   - If equal, jump to `setNextTailLeft` to update tail position leftwards.
7. **Load "snake right" color/direction** into `$t5`.
8. **Compare tail pixel's** `$t4` **with "snake right" code:**
   - If equal, jump to `setNextTailRight` to update tail position rightwards.

```
setNextTailUp:
        addi    $t0, $t0, -256          # tail = tail - 256
        sw      $t0, tail               # store  tail in memory
        j exitUpdateSnake

setNextTailDown:
        addi    $t0, $t0, 256           # tail = tail + 256
        sw      $t0, tail               # store  tail in memory
        j exitUpdateSnake

setNextTailLeft:
        addi    $t0, $t0, -4            # tail = tail - 4
        sw      $t0, tail               # store  tail in memory
        j exitUpdateSnake

setNextTailRight:
        addi    $t0, $t0, 4             # tail = tail + 4
        sw      $t0, tail               # store  tail in memory
        j exitUpdateSnake
```

Explanation

`setNextTailUp:`

- Subtract **256** from the tail address (`$t0 = $t0 - 256`).
- This moves the tail **one row up** in the frame buffer (since 256 bytes correspond to one row).
- Store the updated tail address back to memory.
- Jump to `exitUpdateSnake` to finish.

`setNextTailDown:`

- Add **256** to the tail address (`$t0 = $t0 + 256`).
- This moves the tail **one row down** in the frame buffer.
- Store the updated tail address back to memory.
- Jump to `exitUpdateSnake` .

`setNextTailLeft:`

- Subtract **4** from the tail address (`$t0 = $t0 - 4`).
- This moves the tail **one pixel left** (4 bytes per pixel).
- Store the updated tail address back to memory.
- Jump to `exitUpdateSnake` .

`setNextTailRight:`

- Add **4** to the tail address (`$t0 = $t0 + 4`).
- This moves the tail **one pixel right** (4 bytes per pixel).
- Store the updated tail address back to memory.
- Jump to `exitUpdateSnake` .

```
exitUpdateSnake:

        lw      $ra, 4($sp)     # load caller's return address
        lw      $fp, 0($sp)     # restores caller's frame pointer
        addiu   $sp, $sp, 24    # restores caller's stack pointer
        jr      $ra             # return to caller's code
```

1. **Restore return address**

   `lw $ra, 4($sp)`

   Load the saved return address from the stack into the `$ra` register.

2. **Restore frame pointer**

   `lw $fp, 0($sp)`

   Load the saved frame pointer from the stack into the `$fp` register.

3. **Deallocate stack frame**

   `addiu $sp, $sp, 24`

   Add 24 to the stack pointer to clean up the space allocated for this function's stack frame.

4. **Return to caller**

   `jr $ra`

   Jump to the return address to resume execution after the function call.

10- updateSnakeHeadPosition:

```
updateSnakeHeadPosition:
        addiu   $sp, $sp, -24   # allocate 24 bytes for stack
        sw      $fp, 0($sp)     # store caller's frame pointer
        sw      $ra, 4($sp)     # store caller's return address
        addiu   $fp, $sp, 20    # setup updateSnake frame pointer

        lw      $t3, xVel       # load xVel from memory
        lw      $t4, yVel       # load yVel from memory
        lw      $t5, xPos       # load xPos from memory
        lw      $t6, yPos       # load yPos from memory
        add     $t5, $t5, $t3   # update x pos
        add     $t6, $t6, $t4   # update y pos
        sw      $t5, xPos       # store updated xpos back to memory
        sw      $t6, yPos       # store updated ypos back to memory

        lw      $ra, 4($sp)     # load caller's return address
        lw      $fp, 0($sp)     # restores caller's frame pointer
        addiu   $sp, $sp, 24    # restores caller's stack pointer
        jr      $ra             # return to caller's code
```

Explanation

1. **Function prologue (setup stack frame):**
   - `addiu $sp, $sp, -24` — Allocate 24 bytes on the stack.
   - `sw $fp, 0($sp)` — Save the old frame pointer.
   - `sw $ra, 4($sp)` — Save the return address.
   - `addiu $fp, $sp, 20` — Set up new frame pointer for this function.
2. **Load current velocity and position:**
   - `lw $t3, xVel` — Load current x velocity.
   - `lw $t4, yVel` — Load current y velocity.
   - `lw $t5, xPos` — Load current x position.
   - `lw $t6, yPos` — Load current y position.
3. **Calculate new head position:**
   - `add $t5, $t5, $t3` — Add x velocity to x position.
   - `add $t6, $t6, $t4` — Add y velocity to y position.

4. **Store updated position back:**
   - `sw $t5, xPos` — Store new x position.
   - `sw $t6, yPos` — Store new y position.
5. **Function epilogue (restore stack and return):**
   - `lw $ra, 4($sp)` — Restore return address.
   - `lw $fp, 0($sp)` — Restore frame pointer.
   - `addiu $sp, $sp, 24` — Deallocate stack space.
   - `jr $ra` — Return to the caller.

11- drawApple function

```
drawApple:
        addiu   $sp, $sp, -24   # allocate 24 bytes for stack
        sw      $fp, 0($sp)     # store caller's frame pointer
        sw      $ra, 4($sp)     # store caller's return address
        addiu   $fp, $sp, 20    # setup updateSnake frame pointer

        lw      $t0, appleX             # t0 = xPos of apple
        lw      $t1, appleY             # t1 = yPos of apple
        lw      $t2, xConversion        # t2 = 64
        mult    $t1, $t2                # appleY * 64
        mflo    $t3                     # t3 = appleY * 64
        add     $t3, $t3, $t0           # t3 = appleY * 64 + appleX
        lw      $t2, yConversion        # t2 = 4
        mult    $t3, $t2                # (yPos * 64 + appleX) * 4
        mflo    $t0                     # t0 = (appleY * 64 + appleX) * 4

        la      $t1, frameBuffer        # load frame buffer address
        add     $t0, $t1, $t0           # t0 = (appleY * 64 + appleX) * 4 + frame address
        li      $t4, 0x00ff0000
        sw      $t4, 0($t0)             # store direction plus color on the bitmap display

        lw      $ra, 4($sp)     # load caller's return address
        lw      $fp, 0($sp)     # restores caller's frame pointer
        addiu   $sp, $sp, 24    # restores caller's stack pointer
        jr      $ra             # return to caller's code
```

Explanation

1. **Function prologue (stack setup):**
   - Allocate 24 bytes on the stack: `addiu $sp, $sp, -24`
   - Save caller's frame pointer: `sw $fp, 0($sp)`
   - Save caller's return address: `sw $ra, 4($sp)`
   - Setup this function's frame pointer: `addiu $fp, $sp, 20`

2. **Load Apple Position:**
    - Load apple's X coordinate into `$t0`: `lw $t0, appleX`
    - Load apple's Y coordinate into `$t1`: `lw $t1, appleY`
3. **Calculate pixel offset for the apple on screen:**
    - Load horizontal scaling factor (64): `lw $t2, xConversion`
    - Multiply appleY by 64: `mult $t1, $t2`
    - Move low result of multiplication to `$t3`: `mflo $t3`
    - Add appleX to scaled Y value: `add $t3, $t3, $t0`
    - Load vertical scaling factor (4): `lw $t2, yConversion`
    - Multiply total position by 4 (pixel size): `mult $t3, $t2`
    - Move low result to `$t0`: `mflo $t0`
4. **Calculate memory address of apple pixel in frame buffer:**
    - Load base address of frame buffer: `la $t1, frameBuffer`
    - Add offset to frame buffer base: `add $t0, $t1, $t0`
5. **Draw the apple pixel:**
    - Load apple color (red): `li $t4, 0x00ff0000`
    - Store apple color to pixel address: `sw $t4, 0($t0)`
6. **Function epilogue (restore stack and return):**
    - Restore return address: `lw $ra, 4($sp)`
    - Restore frame pointer: `lw $fp, 0($sp)`
    - Deallocate stack space: `addiu $sp, $sp, 24`
    - Return to caller: `jr $ra`

```
newAppleLocation:
        addiu   $sp, $sp, -24    # allocate 24 bytes for stack
        sw      $fp, 0($sp)      # store caller's frame pointer
        sw      $ra, 4($sp)      # store caller's return address
        addiu   $fp, $sp, 20     # setup updateSnake frame pointer
```

Explanation

1. **Allocate stack space:**
   - `addiu $sp, $sp, -24`
     This reserves 24 bytes on the stack for local variables and saved registers.
2. **Save caller's frame pointer:**
   - `sw $fp, 0($sp)`
     This saves the current frame pointer at the bottom of the new stack frame.
3. **Save caller's return address:**
   - `sw $ra, 4($sp)`
     This saves the return address, so you can return properly after the function ends.
4. **Set up new frame pointer for this function:**
   - `addiu $fp, $sp, 20`
     The frame pointer is set to point near the top of the new stack frame for easy reference to local variables or saved values.

```
redoRandom:
        addi    $v0, $zero, 42  # random int
        addi    $a1, $zero, 63  # upper bound
        syscall
        add     $t1, $zero, $a0 # random appleX

        addi    $v0, $zero, 42  # random int
        addi    $a1, $zero, 31  # upper bound
        syscall
        add     $t2, $zero, $a0 # random appleY

        lw      $t3, xConversion        # t3 = 64
        mult    $t2, $t3                # random appleY * 64
        mflo    $t4                     # t4 = random appleY * 64
        add     $t4, $t4, $t1           # t4 = random appleY * 64 + random appleX
        lw      $t3, yConversion        # t3 = 4
        mult    $t3, $t4                # (random appleY * 64 + random appleX) * 4
        mflo    $t4                     # t1 = (random appleY * 64 + random appleX) * 4

        la      $t0, frameBuffer        # load frame buffer address
        add     $t0, $t4, $t0           # t0 = (appleY * 64 + appleX) * 4 + frame address
        lw      $t5, 0($t0)             # t5 = value of pixel at t0

        li      $t6, 0x00d3d3d3         # load light gray color
        beq     $t5, $t6, goodApple     # if loction is a good sqaure branch to goodApple
        j redoRandom
```

1. **Generate random `appleX` position:**
   - `addi $v0, $zero, 42` — Set syscall code 42 for random int.
   - `addi $a1, $zero, 63` — Set upper bound to 63 (max X coordinate).
   - `syscall` — Generate random int in `$a0` (0 to 63).
   - `add $t1, $zero, $a0` — Store random X position in `$t1`.
2. **Generate random `appleY` position:**
   - `addi $v0, $zero, 42` — Set syscall code 42 again for random int.
   - `addi $a1, $zero, 31` — Set upper bound to 31 (max Y coordinate).
   - `syscall` — Generate random int in `$a0` (0 to 31).
   - `add $t2, $zero, $a0` — Store random Y position in `$t2`.
3. **Calculate pixel offset for `(appleX, appleY)` in frame buffer:**
   - `lw $t3, xConversion` — Load X conversion factor (64).
   - `mult $t2, $t3` — Multiply Y position by 64.
   - `mflo $t4` — Get result into `$t4`.
   - `add $t4, $t4, $t1` — Add X position to Y*64, giving pixel offset.
   - `lw $t3, yConversion` — Load Y conversion factor (4).
   - `mult $t3, $t4` — Multiply pixel offset by 4 (bytes per pixel).
   - `mflo $t4` — Get byte offset in `$t4`.
4. **Check color at calculated position in frame buffer:**
   - `la $t0, frameBuffer` — Load base address of frame buffer.
   - `add $t0, $t4, $t0` — Add byte offset to base address.
   - `lw $t5, 0($t0)` — Load pixel color at that position.
   - `li $t6, 0x00d3d3d3` — Load "light gray" color (valid empty square).
   - `beq $t5, $t6, goodApple` — If pixel color is light gray, it's a valid spot; jump to `goodApple`.
   - `j redoRandom` — Otherwise, try another random position.

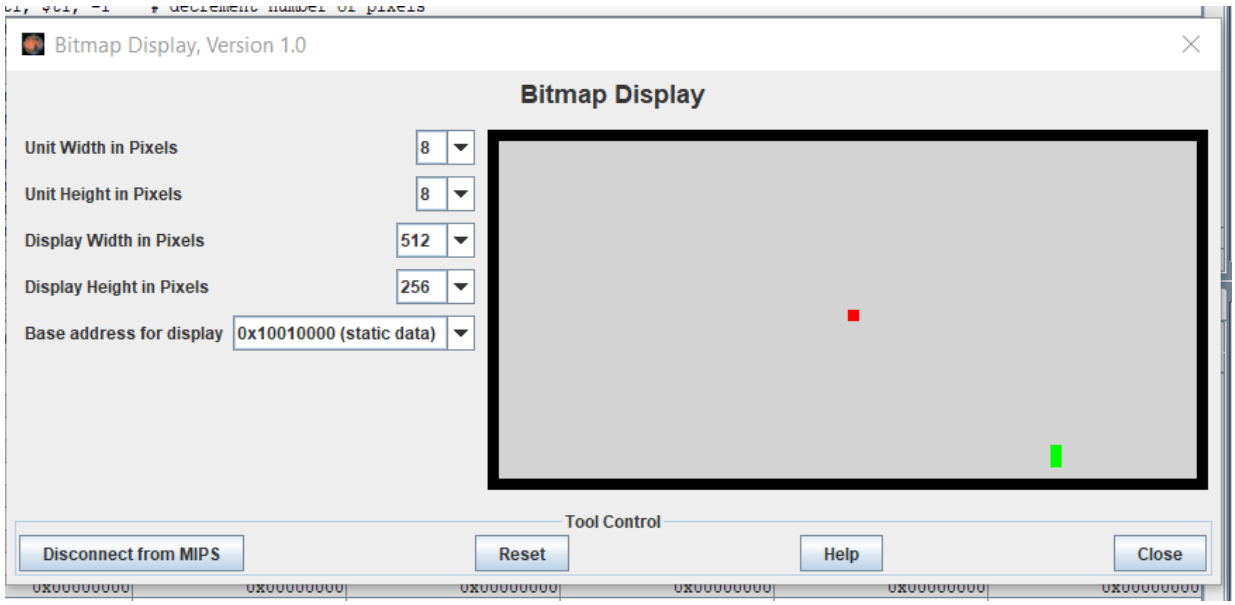
```
goodApple:
        sw      $t1, appleX
        sw      $t2, appleY

        lw      $ra, 4($sp)     # load caller's return address
        lw      $fp, 0($sp)     # restores caller's frame pointer
        addiu   $sp, $sp, 24    # restores caller's stack pointer
        jr      $ra             # return to caller's code
```

Explanation

1. **Store valid apple position:**
   - `sw $t1, appleX` — Save the valid random X coordinate into `appleX`.
   - `sw $t2, appleY` — Save the valid random Y coordinate into `appleY`.
2. **Restore stack and return:**
   - `lw $ra, 4($sp)` — Load return address from stack.
   - `lw $fp, 0($sp)` — Restore frame pointer from stack.
   - `addiu $sp, $sp, 24` — Deallocate 24 bytes from stack (clean up).
   - `jr $ra` — Return control to caller.

Results



Conclusion

This project successfully implements core mechanics of the classic Snake game using MIPS assembly language, demonstrating effective use of low-level programming concepts such as memory management, conditional branching, and stack operations. The program accurately handles snake movement, velocity setting, collision detection, apple placement, and graphical updates on the bitmap display. Through modular subroutines, the code maintains clarity and reusability, allowing for efficient debugging and future enhancements. This exercise highlights the challenges and learning opportunities in developing interactive applications at the hardware-near level, strengthening understanding of computer architecture and assembly programming fundamentals.

Thank You ....